

DATA STRUCTURES AND ALGORITHMS – 3 (25CS2103E)

Mini Search Engine

Information Retrieval System

Submitted by

Surapuraju Himasree (2520090181)

Rachapudi Nishanth (2520090141)

Section 10

Under the guidance of

K. Chandusha, Asst. Prof., CS&IT

A.Y. 2026–2027

Introduction

1

Growing Information

As digital text collections grow rapidly, manually scanning documents to find relevant information becomes slow and impractical.

2

Purpose of the Project

The Mini Search Engine retrieves relevant information from a collection of text documents quickly and accurately.

3

Core Idea

Documents are organized using an indexing mechanism so users can locate information without reading every file.

4

Learning Focus

Core search algorithms are built from scratch, giving hands-on insight into indexing, ranking, and string processing.

Why It Matters

- Fast, accurate, user-friendly search experience
- Supports keyword, phrase, fuzzy, and similarity search
- Ranks results by relevance so top results matter most
- Applies core CS concepts: data structures & algorithms

Abstract

The Mini Search Engine is a software application designed to efficiently search and retrieve relevant information from a collection of text documents. The primary objective of the project is to provide users with a fast, accurate, and user-friendly search experience by implementing different search techniques. The system organizes documents using an efficient indexing mechanism, allowing users to quickly locate the information they need without scanning every document manually. It supports multiple search methods, including exact keyword search, phrase search, fuzzy search for handling misspelled words, and document similarity search to identify documents related to the user's query. The retrieved documents are ranked according to their relevance, ensuring that the most useful results appear first. The project focuses on implementing the core search algorithms from scratch, providing a deeper understanding of information retrieval concepts and efficient search techniques. Appropriate data structures are used to improve search performance, reduce retrieval time, and handle large collections of documents effectively. The efficiency of each algorithm is analysed using time complexity to evaluate its performance. Overall, the Mini Search Engine demonstrates the practical application of data structures, string processing, indexing, and ranking algorithms in developing a simple yet effective document retrieval system, making it a valuable learning project in the field of computer science and software development.

Software Requirements



Java

Core programming language used to implement the search, indexing, and ranking algorithms.



VS Code

Primary code editor / IDE used for development, debugging, and running the application.



Windows OS

Operating system used to develop, build, and run the project.



GitHub

Version control and repository hosting for source code, collaboration, and change tracking.

Hardware Requirements

1

Processor

Intel Core i3 (or equivalent) and above

2

RAM

4 GB minimum, 8 GB recommended

3

Storage

At least 500 MB free disk space for documents, index files, and project setup

4

Display

Standard monitor with minimum 1366 x 768 resolution

Development Environment

- Java Development Kit (JDK)
- Visual Studio Code + Java extensions
- Windows 10 / 11
- Git & GitHub account

Literature Survey



Google Search

Large-scale web search engine using indexing, relevance ranking, and query processing to return the most useful results first.



Apache Lucene

Open-source indexing and search library that pioneered efficient inverted-index techniques for fast full-text retrieval.



Elasticsearch

Distributed search engine built on Lucene, showing how indexing and ranking can scale across large document collections.



Academic Mini-Projects

Prior coursework implementing basic keyword and Boolean search over small document sets, reviewed for design ideas.

Methodology

1

Document Collection

Gather and pre-process the text document set that will be indexed and searched.



2

Text Pre-processing

Tokenize text, remove stop words, and normalize (lowercasing, stemming) to prepare clean terms.



3

Indexing

Build an inverted index mapping each term to the documents and positions where it appears.

4

Query Processing

Parse the user's query and match it against the index using keyword, phrase, or fuzzy matching.



5

Ranking

Score and rank matching documents by relevance using term-weighting and similarity measures.



6

Result Evaluation

Return ranked results to the user and analyse algorithm efficiency using time complexity.

Thank You

Mini Search Engine – Information Retrieval System

Surapuraju Himasree • Rachapudi Nishanth

Section 10